OPTATIVA I · FUNDAMENTOS DE LA INTELIGENCIA ARTIFICIAL EN LA INGENIERÍA DE SOFTWARE

Bitácora de Prompts y Reflexión Técnica

Asignación: Ingeniería de Software Potenciada por IA

Caso de uso: VetSync — Sistema de Gestión de Citas y Telemedicina Veterinaria

Equipo	Jose Maria Quiroz Gastelum Jesus Sergio Aboytia Valenzuela Ahsly Chantal Cota Pacheco
Profesor	Ramon Rene Palacios Cinco
Repositorio	https://github.com/JoseM1809/VetSync
Herramienta de IA	Claude Code (modelo Claude Opus 5.5)
Stack	Node.js 22 · TypeScript · Express · MySQL 8.4 · Jest
Fecha	8 de noviembre de 2026

Contenido

1. Bitácora de prompts (5 prompts propios + Anexo B con los prompts sugeridos)
2. Reflexión técnica
3. Anexo A: respuestas íntegras de la IA a los 5 prompts propios

Bitácora de Prompts — VetSync

- **Asignación:** Ingeniería de Software Potenciada por IA
- **Caso:** VetSync — Gestión de Citas y Telemedicina Veterinaria
- **Herramienta principal:** Claude Code (CLI en terminal), modelo **Claude Opus 5.5**, con acceso al repositorio local (lectura/escritura de archivos y ejecución de `npm`, `jest`, `eslint`, `docker`).

Se listan los **5 prompts más complejos** del proyecto. Todos son prompts propios, diseñados para este caso y ejecutados sobre el código real. Para cada uno se incluye el prompt exacto, la herramienta, un extracto de la respuesta de la IA (el texto íntegro está en el Anexo A) y la crítica humana. Los prompts sugeridos por la actividad también se usaron, en la construcción inicial del sistema; se resumen en el Anexo B.

Estructura de los prompts

Cada prompt sigue la misma estructura, para que la IA no tenga que adivinar el contexto ni el formato:

Elemento	Propósito
Rol	Fija el punto de vista experto (arquitecto, QA, AppSec, soporte, diseño de APIs)
Contexto	Stack, archivos relevantes y estado del sistema, para que la respuesta se base en el código real
Tarea	Qué se pide, en pasos numerados
Restricciones	Qué no debe hacer (inventar, modificar producción, matar procesos), para reducir alucinaciones
Formato de salida	Secciones o artefacto exacto, para poder verificar la respuesta

Resumen

#	Fase	Prompt	Resultado clave de la revisión humana
1	I – Arquitectura	ADR: estrategias contra la doble reserva	Un experimento demostró que la opción recomendada era frágil: una lectura previa reabre la doble reserva
2	II – TDD / QA	Partición de equivalencia y valores límite	32 casos; 5 fallaron en el código real (30 de febrero se convertía en 2 de marzo)
3	II – Caza de bugs	Diagnóstico de un `EADDRINUSE` real	Causa confirmada con evidencia; la IA detectó un problema pero su código no lo corregía
4	III – Seguridad	STRIDE + OWASP API Top 10	Recomendó `timingSafeEqual` sin advertir que lanza excepción si las longitudes difieren
5	III – Documentación	Contrato OpenAPI 3.1 desde el código	La especificación "completa" no pasó el linter (14 errores → 0 tras la corrección); se creó un test que detecta rutas inventadas

---

Prompt 1 — Decisión de arquitectura: estrategia contra la doble reserva

Prompt exacto

> Actúa como Arquitecto de Software Principal especializado en sistemas transaccionales. **Contexto:** VetSync es un backend Node.js 22 + TypeScript + MySQL 8.4 (InnoDB, aislamiento por defecto) organizado como monolito modular hexagonal. La regla de negocio crítica es: un veterinario no puede tener dos citas en estado PROGRAMADA cuyos intervalos [inicio, fin) se solapen, aunque las peticiones lleguen al mismo tiempo. Las citas duran entre 1 y 240 minutos y pueden empezar en cualquier minuto. **Tarea:** redacta un Architecture Decision Record en formato MADR que compare al menos cuatro estrategias para garantizar esa invariante. **Para cada opción indica:** el mecanismo exacto en MySQL, el nivel de aislamiento que requiere, qué recibe el segundo cliente cuando dos peticiones compiten, el impacto en rendimiento y cómo se probaría. **Restricciones:** no propongas cambiar de motor de base de datos; cita únicamente comportamientos documentados de InnoDB. **Formato de salida:** Markdown con las secciones Contexto, Factores de decisión, Opciones, Comparación (tabla), Decisión y Consecuencias.

Herramienta: Claude Code (Claude Opus 5.5).

Respuesta de la IA (extracto) — texto completo en el Anexo A.1

- Comparó 5 opciones: (A) candado `SELECT … FOR UPDATE` por veterinario, (B) aislamiento `SERIALIZABLE`, (C) tabla de franjas con `UNIQUE`, (D) candados con nombre `GET_LOCK` y (E) concurrencia optimista con versión.
- Afirmó que con `SERIALIZABLE` el perdedor recibe un *deadlock* 1213 y que la opción A responde 409 sin reintentos.
- Recomendó la opción A con esta advertencia: "*La consulta de solapamiento DEBE ejecutarse dentro de la misma transacción y después del* `FOR UPDATE`."
- Sobre `GET_LOCK`: "*si el proceso cae con la conexión viva el candado sigue tomado*".

Crítica — qué se cambió, por qué y qué alucinaciones hubo

Hallazgo	Tipo	Cómo se detectó	Decisión
`SERIALIZABLE` produce deadlock 1213	Correcto	Experimento con dos transacciones reales en MySQL 8.4: una recibió `ER_LOCK_DEADLOCK`.	Se descartó la opción B.
Opción A con `FOR UPDATE` como primera sentencia	Correcto	Experimento: 1 cita guardada, la otra transacción recibe conflicto.	Se mantuvo.
La advertencia "después del `FOR UPDATE`" es cierta, pero la IA la dejó como **regla de disciplina** sin medir el riesgo	Riesgo subestimado	Experimento A-bis: con una sola lectura previa al candado, **las dos transacciones guardaron la cita**. InnoDB fija la "foto" de los datos en la primera lectura simple.	La consulta de solapamiento se volvió lectura con candado (`… LIMIT 1 FOR SHARE`), que siempre ve lo último confirmado. Verificado: 1 cita y 409 incluso con lectura previa; integración 3 veces en verde.

Hallazgo	Tipo	Cómo se detectó	Decisión
`GET_LOCK` "con el proceso caído y la conexión viva"	Afirmación contradictoria	Lectura crítica: si el proceso cae, la conexión se cierra y MySQL libera el candado. El riesgo real es devolver la conexión al pool sin `RELEASE_LOCK`.	Corregido en el ADR final.

Resultado: `docs/adr/0001-evitar-doble-reserva.md` (aceptado), con una tabla de 5 experimentos y un cambio de una línea en `repositorioCitasMySQL.ts`.

Prompt 2 — Diseño de pruebas: partición de equivalencia y valores límite

Prompt exacto

> Actúa como Ingeniero de QA especializado en diseño de casos de prueba. **Contexto:** en VetSync, `POST /api/citas` recibe JSON; la capa HTTP lo convierte con `leerFecha`, `leerEnteroPositivo` y `leerModalidad` (`src/compartido/http.ts`, `src/modulos/citas/http/rutasCitas.ts`) y después `ServicioCitas.agendarCita` aplica las reglas de negocio (`src/modulos/citas`). Usa un reloj fijo en `2026-11-10T15:00:00Z`. **Tarea:** aplica partición de equivalencia y análisis de valores límite a los campos `inicio`, `duracionMinutos`, `mascotaId`, `veterinarioId`, `modalidad` y `motivo`. **Entregables:** 1) una matriz con clase de equivalencia, valor representativo y código HTTP esperado según el contrato de la API; 2) la lista de casos que el código actual probablemente maneja mal, con el valor exacto que lo demuestra; 3) un archivo de pruebas Jest + supertest parametrizado con `it.each` que cubra la matriz. **Restricciones:** no modifiques código de producción; cada caso debe cambiar un solo campo respecto de una solicitud válida.

Herramienta: Claude Code (Claude Opus 5.5).

Respuesta de la IA (extracto) — texto completo y código en el Anexo A.2

- Matriz de 32 casos (12 de `inicio`, 7 de duración, 6 de ids, 3 de modalidad, 4 de motivo).
- Cuatro sospechas: `Date.parse` convierte `2027-02-30` en el 2 de marzo; lo mismo con el 29 de febrero de un año no bisiesto; no hay horizonte máximo (se acepta el año 9999); el cuerpo acepta números como texto (`"30"`).
- Archivo `tests/integracion/particionCitas.test.ts` con `it.each`.

Crítica

Hallazgo	Tipo	Cómo se detectó	Decisión
Las 4 sospechas eran reales	Acierto	Al ejecutar los tests: **5 fallos exactamente en esos casos** (27 de 32 pasaban). Una cita del "30 de febrero" se habría agendado el 2 de marzo sin avisar.	`leerFecha` verifica que la fecha exista; horizonte de 365 días configurable en `ILimitesAgenda`; `leerEnteroDelCuerpo` exige números JSON. Resultado: 32/32.
Los títulos de los tests decían `→ HTTP NaN`	Error en el código generado	Salida de Jest: el orden de la tupla `[nombre, cambios, esperado]` no coincidía con el formato `'%s → HTTP %i'`. Los tests evaluaban bien, pero el reporte era ilegible.	Se reordenó la tupla a `[nombre, esperado, cambios]`.

Hallazgo	Tipo	Cómo se detectó	Decisión
Faltaba el caso válido del 29 de febrero bisiesto y el borde exacto del horizonte	Omisión	Revisión de la matriz: sin ellos, la corrección del calendario podía romper fechas válidas sin que nada fallara.	Tests unitarios: `2028-02-29` aceptado; 365 días aceptado, 365 días + 1 minuto rechazado.
Respetó la restricción de no tocar producción	Correcto	Revisión del diff.	—

Prompt 3 — Caza de bugs: diagnóstico de un fallo real al arrancar

Prompt exacto

> Actúa como Ingeniero de Soporte de Producción experto en Node.js sobre Windows. **Contexto:** al ejecutar `npm start` para VetSync (Windows 11, Node 24, puerto 3010 configurado en `.env`) el proceso terminó con la salida de abajo. Minutos antes, el mismo servidor se había ejecutado como tarea en segundo plano y esa tarea se detuvo.
>
> ```
> Error: listen EADDRINUSE: address already in use :::3010
>     at Server.setupListenHandle [as _listen2] (node:net:2167:16)
>     at app.listen (…\node_modules\express\lib\application.js:635:24)
>     at Object.<anonymous> (…\dist\servidor.js:13:22)
> Emitted 'error' event on Server instance at: …
> ```
>
> **Tarea:** 1) indica las 3 causas raíz más probables, ordenadas, con el comando exacto de PowerShell para confirmar o descartar cada una; 2) explica por qué el proceso termina con un stack trace en lugar de un mensaje claro; 3) propone el cambio mínimo en `src/servidor.ts` para que este fallo sea diagnosticable sin ocultar el error. **Restricción:** no sugieras terminar procesos de forma indiscriminada (por ejemplo, todos los `node.exe`).

Herramienta: Claude Code (Claude Opus 5.5).

Respuesta de la IA (extracto) — texto completo en el Anexo A.3

- Causas: (1) la instancia anterior quedó huérfana, porque en Windows detener el proceso padre no siempre termina al hijo `node`; (2) otra aplicación usa el puerto; (3) un rango de puertos reservado por Hyper-V/Docker, aclarando que eso suele dar `EACCES` y no `EADDRINUSE`.
- Comandos `Get-NetTCPConnection` + `Get-CimInstance Win32_Process` para identificar el PID, y terminar sólo ese PID.
- Causa del stack trace: el `http.Server` emite `'error'` sin ningún *listener*. Propuso `servidor.on('error', …)` con un mensaje accionable y `process.exit(1)`.
- Observó que el job de recordatorios se programa antes de saber si el servidor arrancó.

Crítica

Hallazgo	Tipo	Cómo se detectó	Decisión
Causa 1	Correcta y confirmada	Evidencia de esta misma sesión: el comando propuesto mostró el PID 32492 con `node dist/servidor.js`, huérfano de la tarea detenida. Se terminó **sólo** ese PID.	—
La IA detectó que el job arranca aunque el servidor falle, pero **su código propuesto no lo corrige**	Hueco entre diagnóstico y solución	Lectura del código sugerido: el `setInterval` seguía fuera del callback de `listen`.	El job se programa dentro del callback de `listen`, es decir, sólo si el servidor quedó escuchando.
El cambio propuesto funciona	Verificado	Se ocupó el puerto 3011 con un proceso de prueba y se arrancó VetSync ahí: mensaje claro con el comando de diagnóstico y código de salida 1.	Aplicado.
Causa 3 (`EACCES` vs `EADDRINUSE`)	No verificada	No se reprodujo un rango reservado en esta sesión.	Se registra como afirmación no comprobada.

Prompt 4 — Seguridad: modelado de amenazas STRIDE + OWASP API Top 10

Prompt exacto

> Actúa como especialista en Seguridad de Aplicaciones (AppSec). **Contexto:** VetSync es una API REST (Express 4 + MySQL) que maneja datos personales de dueños y el historial médico de sus mascotas. Por decisión de alcance de la práctica no tiene autenticación de usuarios. El código está en `src/` y las rutas en `src/modulos/*/http`. **Tarea:** realiza un modelado de amenazas STRIDE de la API y relaciona cada amenaza con el OWASP API Security Top 10 (2023). **Formato de salida:** una tabla priorizada con: amenaza, categoría STRIDE, riesgo OWASP API, evidencia (archivo y fragmento de código que la origina), severidad y mitigación. Después, separa (a) mitigaciones implementables hoy sin dependencias nuevas y en pocas líneas, y (b) las que requieren una decisión de producto. **Restricciones:** no reportes ninguna vulnerabilidad sin señalar el código que la origina; marca como 'no aplica' las categorías del Top 10 que no correspondan y explica por qué.

Herramienta: Claude Code (Claude Opus 5.5).

Respuesta de la IA (extracto) — texto completo en el Anexo A.4

- 9 amenazas priorizadas. Las más graves: lectura de historiales por ids secuenciales (API1), cancelar citas ajenas (API1), y rutas de notificaciones abiertas que exponen el email de todos los dueños (API5).
- Hallazgo no obvio: un nombre con `\r\n` llega al asunto del correo, lo que permitiría inyectar cabeceras al conectar un SMTP real.
- (a) Implementables hoy: token de administrador comparado con `crypto.timingSafeEqual` ("*si `TOKEN_ADMIN` no está configurado, esas rutas responden 401 siempre*"), rechazo de caracteres de control y cabeceras de seguridad. (b) Decisiones de producto: autenticación con roles, rate limiting, límite de

citas por mascota y versionado.

Crítica

Hallazgo	Tipo	Cómo se detectó	Decisión
Cada amenaza señala código real	Correcto	Se revisaron las 9 contra el código: todas existen.	—
`timingSafeEqual` **lanza una excepción si las longitudes difieren**; la IA no lo advirtió	Omisión técnica	Prueba directa en Node: `ERR_CRYPTO_TIMING_SAFE_EQUAL_LENGTH`. Implementado tal cual, un token de otra longitud habría producido un **500** en vez de un 401.	Se comparan huellas SHA-256 (siempre 32 bytes). Test: "token de otra longitud → 401".
"Sin token configurado responde 401"	Código HTTP impreciso	401 sugiere que otra credencial funcionaría, pero ninguna puede funcionar.	Se usa **403** (ruta deshabilitada).
Expresión regular de caracteres de control	Problema al implementar	ESLint (`no-control-regex`) la rechazó.	En lugar de desactivar la regla, se validó por código de carácter.
Mitigaciones de tipo (b)	Decisión	Fuera del alcance de la práctica.	Documentadas en el README como limitaciones.

Resultado: `src/compartido/seguridad.ts` y 10 tests de seguridad (cabeceras, 401/403, inyección de saltos de línea).

Prompt 5 — Documentación: contrato OpenAPI 3.1 a partir del código

Prompt exacto

> Actúa como Arquitecto de APIs especializado en diseño *contract-first*. **Contexto:** VetSync es una API Express; las rutas están en `src/modulos/*/http`, la validación de entrada en `src/compartido/http.ts` y `src/compartido/validacion.ts`, los errores en `src/compartido/errores.ts`, la seguridad en `src/compartido/seguridad.ts` y las reglas de citas en `src/modulos/citas/dominio/politicaAgendamiento.ts`. **Tarea:** genera la especificación OpenAPI 3.1 completa (`docs/openapi.yaml`) que describa exactamente el comportamiento implementado: rutas, parámetros, cuerpos con tipos y restricciones (longitudes, rangos, formatos, enumeraciones), respuestas con todos sus códigos, el esquema de error común y el esquema de seguridad de las rutas de administración. **Restricciones:** no documentes campos ni códigos de estado que el código no implemente; si el código y `docs/API.md` discrepan, gana el código y enumera las discrepancias al final. **Formato de salida:** YAML válido con componentes reutilizables en `components`.

Herramienta: Claude Code (Claude Opus 5.5).

Respuesta de la IA (extracto) — especificación completa en el Anexo A.5

- 16 operaciones, 16 esquemas, 6 respuestas reutilizables y el esquema de seguridad `X-Token-Admin`.

- 6 discrepancias con `API.md`: faltaban los códigos 401/403 y el token, el horizonte de 365 días, la validación de calendario, los enteros como números JSON y la regla de caracteres de control.

Crítica

Hallazgo	Tipo	Cómo se detectó	Decisión
La especificación se veía completa pero **no era válida**	Error	Redocly CLI: **14 errores y 9 advertencias**. Las advertencias visibles: 6 etiquetas sin descripción y 2 operaciones sin respuesta 4xx.	Se agregaron descripciones a las etiquetas y `security: []` a las 14 operaciones públicas (los 14 errores coincidían en número). **Segunda ejecución del linter: 0 errores** ("Your API description is valid"), lo que confirma la causa. Quedan 3 advertencias aceptadas: el servidor `localhost` (es el de desarrollo) y las dos de abajo.
Advertencia "operación sin respuesta 4xx" en `GET /salud` y `GET /veterinarios`	Decisión	Esas rutas no producen errores 4xx.	Se aceptan las advertencias: agregar un 4xx sería documentar algo falso, lo que la propia restricción prohíbe.
Las 6 discrepancias	Correctas	Cada una se verificó contra el código.	`docs/API.md` actualizado.
Riesgo de rutas alucinadas	Prevención	Se creó `tests/integracion/contratoOpenapi.test.ts`: compara las operaciones del YAML con las rutas del código y llama a cada una. **Prueba de mutación:** al agregar una ruta inventada (`/citas/{id}/reprogramar`), el test falla.	Ninguna ruta inventada en la especificación real (17/17).

Anexo B — Prompts sugeridos por la actividad (construcción inicial)

Se usaron para construir la primera versión del sistema. Las respuestas íntegras están en `docs/bitacora/respuestas/01` a `07`.

#	Fase	Prompt sugerido	Hallazgo principal de la revisión
S1	I	"Actúa como un Arquitecto de Software Senior… propón un esquema de base de datos relacional (MySQL) y la estructura de directorios…"	Omitió la concurrencia y las zonas horarias; usó `DATETIME` sin milisegundos (MySQL los redondeaba).
S2	I	"Genera un archivo agents.md estricto para un proyecto en [Lenguaje]…"	La regla "try/catch en toda función async" convertiría errores 500 en 400; se cambió a "try/catch en los bordes" y se reforzó con ESLint.

#	Fase	Prompt sugerido	Hallazgo principal de la revisión
S3	II	"Escribe 5 casos de prueba unitarios usando Jest para la función 'agendarCita'..."	Faltaban 10 casos de borde, entre ellos la cita que envuelve a otra.
S4	II	"Revisa este bloque de código. Identifica violaciones a los principios SOLID..."	Correcto en DIP/SRP/OCP; se rechazó un *spread* que filtraba un campo extra al JSON.
S5	III	"Actúa como un experto en Ciberseguridad y Rendimiento..."	Detectó la doble reserva (20 de 20 guardadas en MySQL real); un falso positivo y una alternativa engañosa.
S6	III	"Genera el YAML para un GitHub Action que se ejecute en cada push a 'main'..."	Se verificó que el umbral de cobertura del 80 % sí rompe el build.
S7	III	"Actúa como un Ingeniero de Software Principal y Arquitecto de Soluciones experto en documentación técnica..."	Al documentar se descubrió que `motivo` no se validaba.

Reflexión técnica: carga cognitiva y deuda técnica con IA

¿Dónde la IA redujo la carga cognitiva?

La reducción más clara fue en el trabajo **mecánico y de bajo riesgo**: repositorios, mapeo de filas `snake_case` a `camelCase`, rutas Express, el YAML de la CI y la especificación OpenAPI. Delegarlo nos dejó la atención libre para el dominio.

También ayudó a **explorar más rápido de lo que podríamos solos**. El prompt de QA produjo en minutos una matriz de 32 casos límite y detectó algo que no habríamos buscado: JavaScript convierte el "30 de febrero" en el 2 de marzo, así que la API agendaba citas en otro día sin avisar. El ADR comparó cinco estrategias de concurrencia, un punto de partida mucho mejor que una hoja en blanco.

Diseñar prompts con rol, contexto, tarea, restricciones y formato de salida también ayudó: cuanto más preciso el encargo, más fácil verificar la respuesta.

¿Dónde la aumentó?

Paradójicamente, donde la IA sonaba **más segura**. Cada respuesta llegó bien redactada, y eso obliga a distinguir lo correcto de lo plausible:

- El ADR acertó con la estrategia, pero dejó como simple advertencia algo que un experimento demostró peligroso: una sola lectura antes del candado reabría la doble reserva.
- El análisis de seguridad recomendó `timingSafeEqual` sin decir que lanza una excepción con longitudes distintas; tal cual, un token incorrecto habría dado un error 500.
- La especificación OpenAPI parecía completa y el linter le encontró 14 errores.
- En el diagnóstico del `EADDRINUSE`, la IA vio el problema del job de recordatorios, pero su código no lo corregía.

Detectar una **omisión** cuesta más que corregir un error visible, y también aumentó la carga de **verificar los propios tests**: una prueba de concurrencia pasó en verde con el bug presente, y otra mostraba "HTTP NaN" en sus títulos. Con IA es tentador leer "PASS" como "correcto".

¿Cómo garantizamos que el código generado no introdujera deuda técnica?

1. **Reglas ejecutables, no sólo escritas.** AGENTS.md define convenciones; ESLint y TypeScript `strict` las imponen. Cuando ESLint rechazó una expresión regular, se reescribió el código en vez de desactivar la regla.
2. **TDD como contrato.** Cada corrección empezó con un test en rojo. Hoy son 127 pruebas y 99 % de cobertura de líneas.
3. **Verificación contra el sistema real.** Las afirmaciones de la IA sobre MySQL se comprobaron con transacciones simultáneas reales, no por confianza.
4. **Tests que vigilan a la IA.** Un test de contrato compara la especificación OpenAPI con las rutas del código; con una ruta inventada a propósito, falla.

5. **Puerta automática en la CI**: lint, compilación, tests y cobertura mínima del 80 % en cada push.
6. **Deuda explícita.** Lo que se decidió no hacer (autenticación, rate limiting) quedó documentado con su motivo.

La conclusión: la IA acelera la *escritura* de código, pero no la *responsabilidad* sobre él. El humano decide qué es correcto y prueba que lo es.

Anexo A — Respuestas íntegras de la IA

Texto tal como lo generó la IA para cada uno de los 5 prompts propios, antes de la revisión humana. La crítica de cada respuesta está en la bitácora. Las respuestas a los prompts sugeridos (Anexo B) están en el repositorio, en `docs/bitacora/respuestas/01` a `07`.

A.1 Respuesta de la IA — Prompt 1: ADR: estrategia contra la doble reserva

> **Herramienta:** Claude Code (modelo Claude Opus 5.5), en terminal, con acceso al repositorio. **Prompt exacto:**
>
> "Actúa como Arquitecto de Software Principal especializado en sistemas transaccionales. **Contexto:** VetSync es un backend Node.js 22 + TypeScript + MySQL 8.4 (InnoDB, aislamiento por defecto) organizado como monolito modular hexagonal. La regla de negocio crítica es: un veterinario no puede tener dos citas en estado PROGRAMADA cuyos intervalos [inicio, fin) se solapen, aunque las peticiones lleguen al mismo tiempo. Las citas duran entre 1 y 240 minutos y pueden empezar en cualquier minuto. **Tarea:** redacta un Architecture Decision Record en formato MADR que compare al menos cuatro estrategias para garantizar esa invariante. **Para cada opción indica:** el mecanismo exacto en MySQL, el nivel de aislamiento que requiere, qué recibe el segundo cliente cuando dos peticiones compiten, el impacto en rendimiento y cómo se probaría. **Restricciones:** no propongas cambiar de motor de base de datos; cita únicamente comportamientos documentados de InnoDB. **Formato de salida:** Markdown con las secciones Contexto, Factores de decisión, Opciones, Comparación (tabla), Decisión y Consecuencias."
>
> Texto de la respuesta tal como la generó la IA (sin editar).

ADR-0001: Estrategia para impedir la doble reserva de un veterinario

- **Estado:** propuesto
- **Fecha:** 2026-11-08

Contexto

`agendarCita` verifica si existe una cita solapada y luego inserta. Ejecutadas por separado, dos transacciones concurrentes pueden leer "libre" y ambas insertar (*time-of-check to time-of-use*). MySQL no tiene restricciones de exclusión sobre rangos (como `EXCLUDE USING gist` de PostgreSQL), y un `UNIQUE (veterinario_id, inicio)` no detecta solapamientos parciales, así que la invariante debe imponerse con control de concurrencia.

Factores de decisión

1. Corrección bajo concurrencia (cero dobles reservas).
2. Respuesta limpia al perdedor: un **409 Conflicto**, no un 500.
3. No bloquear reservas de **otros** veterinarios.
4. Soportar duraciones arbitrarias (1–240 min, cualquier minuto de inicio).
5. Poder probarlo de forma automatizada.

Opciones

Opción A — Candado pesimista por veterinario (`SELECT … FOR UPDATE`)

Dentro de una transacción (REPEATABLE READ, el valor por defecto): `SELECT id FROM veterinarios WHERE id = ? FOR UPDATE`, luego la consulta de solapamiento y el `INSERT`; `COMMIT`. La segunda transacción **espera** en el `FOR UPDATE` hasta que la primera confirme; al continuar ve la cita ya insertada y responde 409. La espera máxima la fija `innodb_lock_wait_timeout` (50 s por defecto). Sólo serializa a un mismo veterinario.

Opción B — Aislamiento `SERIALIZABLE`

`SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`. InnoDB convierte los `SELECT` simples en lecturas con candado compartido (*next-key locks*) sobre el rango leído. Dos transacciones que leen el mismo rango vacío obtienen ambas el candado compartido; cuando las dos intentan insertar en ese hueco, cada una espera a la otra → **deadlock** (error 1213, `ER_LOCK_DEADLOCK`). InnoDB revierte una. El perdedor recibe un error que hay que capturar y **reintentar** para poder devolver 409.

Opción C — Tabla de franjas con `UNIQUE`

Discretizar el tiempo en franjas (p. ej. de 5 min) y crear `franjas_ocupadas (veterinario_id, inicio_franja, cita_id)` con `UNIQUE (veterinario_id, inicio_franja)`. Al reservar se insertan todas las franjas de la cita en la misma transacción; si alguna ya existe, MySQL devuelve `ER_DUP_ENTRY` (1062) → 409. La base de datos impone la invariante sin depender del código. Desventajas: obliga a que inicio y duración sean múltiplos de la franja, multiplica filas (una cita de 4 h = 48 filas) y cancelar exige borrar sus franjas.

Opción D — Candados con nombre (`GET_LOCK`)

`SELECT GET_LOCK(CONCAT('vet:', ?), 5)` antes de verificar e insertar, `RELEASE_LOCK` al terminar. Es un candado de **sesión**, no de transacción: debe liberarse en la misma conexión del pool, y si el proceso cae con la conexión viva el candado sigue tomado. Sólo protege a quien también lo pida: otra vía de escritura que no llame a `GET_LOCK` lo ignora.

Opción E — Concurrencia optimista con versión

Columna `version` en `veterinarios`. Leer la versión, verificar solapamientos, insertar y `UPDATE veterinarios SET version = version + 1 WHERE id = ? AND version = ?`; si afecta 0 filas, alguien reservó antes → `ROLLBACK` y reintento (o 409). Sin esperas, pero con reintentos bajo contención.

Comparación

Criterio	A. FOR UPDATE	B. SERIALIZABLE	C. Franjas UNIQUE	D. GET_LOCK	E. Optimista
Cero dobles reservas	Sí	Sí	Sí (impuesto por la BD)	Sólo si todos lo usan	Sí
Perdedor recibe	Espera → 409	Deadlock 1213 → reintento	1062 → 409	Espera → 409	0 filas → reintento/409
Bloquea a otros veterinarios	No	Puede (candados de rango)	No	No	No
Duración libre	Sí	Sí	**No** (múltiplos de franja)	Sí	Sí
Complejidad	Baja	Media (reintentos)	Alta (franjas, cancelación)	Media (sesión/pool)	Media (reintentos)
Prueba	N peticiones paralelas: 1 éxito	Ídem + reintentos	Ídem	Ídem	Ídem

Decisión

Se elige la **Opción A**. Cumple los cinco factores con el menor código, responde 409 sin reintentos y aísla la contención por veterinario.

Consecuencias

- Las reservas de un mismo veterinario se serializan; con volúmenes de una clínica (decenas de citas por hora) la espera es de milisegundos.
- La consulta de solapamiento DEBE ejecutarse dentro de la misma transacción y después del `FOR UPDATE`.
- Si en el futuro se necesitara una garantía impuesta por la base de datos (p. ej. escrituras desde otros sistemas), migrar a la Opción C.
- Prueba de aceptación: 20 reservas simultáneas del mismo horario → exactamente 1 éxito.

A.2 Respuesta de la IA — Prompt 2: Partición de equivalencia y valores límite

> **Herramienta:** Claude Code (modelo Claude Opus 5.5), en terminal, con acceso al repositorio. **Prompt exacto:**
>
> "Actúa como Ingeniero de QA especializado en diseño de casos de prueba. **Contexto:** en VetSync, `POST /api/citas` recibe JSON; la capa HTTP lo convierte con `leerFecha`, `leerEnteroPositivo` y `leerModalidad` (`src/compartido/http.ts`, `src/modulos/citas/http/rutasCitas.ts`) y después `ServicioCitas.agendarCita` aplica las reglas de negocio (`src/modulos/citas`). Usa un reloj fijo en `2026-11-10T15:00:00Z`. **Tarea:** aplica partición de equivalencia y análisis de valores límite a los campos `inicio`, `duracionMinutos`, `mascotaId`, `veterinarioId`, `modalidad` y `motivo`. **Entregables:** 1) una matriz con clase de equivalencia, valor representativo y código HTTP esperado según el contrato de la API; 2) la lista de casos que el código actual probablemente maneja mal, con el valor exacto que lo demuestra; 3) un archivo de pruebas Jest + supertest parametrizado con `it.each` que cubra la matriz. **Restricciones:** no modifiques código de producción; cada caso debe cambiar un solo campo respecto de una solicitud válida."
>
> Texto de la respuesta tal como la generó la IA (sin editar).

---

1. Matriz de partición

Campo	Clase válida (esperado 201)	Clases inválidas (esperado 400/404)
`inicio`	ISO con `Z`; ISO con offset; sólo fecha; 1 min después de "ahora"	igual a "ahora"; sin zona; formato no ISO; mes 13; **fecha inexistente (30-feb)**; **29-feb en año no bisiesto**; **año 9999**; número epoch
`duracionMinutos`	1 (mínimo); 240 (máximo)	0; 241; 12.5; **texto "30"**; null
`mascotaId` / `veterinarioId`	entero existente	0; negativo; > 2^53; **texto "1"**; inexistente (404)
`modalidad`	omitida; `TELEMEDICINA`	minúsculas; null
`motivo`	255 caracteres (máximo)	256 caracteres; vacío; numérico

2. Casos que el código actual probablemente maneja mal

1. **Fechas de calendario inexistentes.** `leerFecha` valida el formato con una expresión regular y luego confía en `Date.parse`, que en V8 *normaliza* fechas imposibles: `2027-02-30T10:00:00Z` se convierte en el 2 de marzo. La cita se agendaría en otro día sin avisar al cliente.
2. **29 de febrero en año no bisiesto** (`2027-02-29`): mismo mecanismo, se convierte en 1 de marzo.
3. **Sin horizonte máximo.** Nada impide agendar en `9999-12-31`. Un error de captura (2206 en lugar de 2026) crearía una cita imposible de atender.
4. **Números enviados como texto.** `leerEnteroPositivo` acepta cadenas de dígitos (necesario para los parámetros de ruta como `/citas/:id`), pero eso hace que el cuerpo JSON acepte `"duracionMinutos": "30"` o `"mascotaId": "1"`, contradiciendo el tipo `integer` del contrato.

3. Pruebas generadas (`tests/integracion/particionCitas.test.ts`)

```
import request from 'supertest';
import { Express } from 'express';
```

```
import { crearApp } from '../../src/app';
import { cargarConfiguracion } from '../../src/compartido/configuracion';
import { registroSilencioso } from '../../src/compartido/registro';
import { construirContenedor } from '../../src/contenedor';
import { crearRelojFijo } from '../apoyo/dobles';

/**
 * Partición de equivalencia y valores límite de POST /api/citas.
 * Reloj fijo: 2026-11-10T15:00:00Z. Cada caso parte de una solicitud válida y cambia UN campo.
 */
type ICampos = Record<string, unknown>;

const BASE: ICampos = { inicio: '2026-11-11T10:00:00Z', duracionMinutos: 30, motivo: 'Consulta' };

const casos: [string, ICampos, number][] = [
  // ---- inicio ----
  ['inicio válido con Z', {}, 201],
  ['inicio con offset -07:00', { inicio: '2026-11-11T10:00:00-07:00' }, 201],
  ['inicio sólo con fecha (medianoche UTC)', { inicio: '2026-11-12' }, 201],
  ['inicio 1 minuto después de "ahora" (límite inferior válido)', { inicio: '2026-11-10T15:01:00Z' }, 201],
  ['inicio igual a "ahora" (límite inferior inválido)', { inicio: '2026-11-10T15:00:00Z' }, 400],
  ['inicio sin zona horaria', { inicio: '2026-11-11T10:00:00' }, 400],
  ['inicio en formato no ISO', { inicio: '11/11/2026 10:00' }, 400],
  ['inicio con mes 13', { inicio: '2026-13-01T10:00:00Z' }, 400],
  ['inicio 30 de febrero (fecha inexistente)', { inicio: '2027-02-30T10:00:00Z' }, 400],
  ['inicio 29 de febrero en año no bisiesto', { inicio: '2027-02-29T10:00:00Z' }, 400],
  ['inicio en el año 9999 (fuera de horizonte)', { inicio: '9999-12-31T10:00:00Z' }, 400],
  ['inicio como número (epoch)', { inicio: 1794405600000 }, 400],
  // ---- duracionMinutos ----
  ['duración 1 (mínimo)', { duracionMinutos: 1 }, 201],
  ['duración 240 (máximo)', { duracionMinutos: 240 }, 201],
  ['duración 0', { duracionMinutos: 0 }, 400],
  ['duración 241', { duracionMinutos: 241 }, 400],
  ['duración decimal', { duracionMinutos: 12.5 }, 400],
  ['duración como texto "30"', { duracionMinutos: '30' }, 400],
  ['duración null', { duracionMinutos: null }, 400],
  // ---- ids ----
  ['mascotaId 0', { mascotaId: 0 }, 400],
```

Fragmento: líneas 1–40 de 75. El texto completo de esta respuesta está en el repositorio: `docs/bitacora/respuestas/09-particion-valores-limite.md`.

Resultado de ejecutarlas sobre el código actual

```
Tests:       5 failed, 27 passed, 32 total
  × inicio 30 de febrero (fecha inexistente)
  × inicio 29 de febrero en año no bisiesto
  × inicio en el año 9999 (fuera de horizonte)
  × duración como texto "30"
  × mascotaId como texto "1"
```

A.3 Respuesta de la IA — Prompt 3: Diagnóstico de un fallo real al arrancar

> **Herramienta:** Claude Code (modelo Claude Opus 5.5), en terminal, con acceso al repositorio. **Prompt exacto:**
>
> "Actúa como Ingeniero de Soporte de Producción experto en Node.js sobre Windows. **Contexto:** al ejecutar `npm start` para VetSync (Windows 11, Node 24, puerto 3010 configurado en `.env`) el proceso terminó con la salida de abajo. Minutos antes, el mismo servidor se había ejecutado como tarea en segundo plano y esa tarea se detuvo.
>
> ```
> node:events:487
>       throw er; // Unhandled 'error' event
>       ^
> Error: listen EADDRINUSE: address already in use :::3010
>     at Server.setupListenHandle [as _listen2] (node:net:2167:16)
>     at app.listen (…\node_modules\express\lib\application.js:635:24)
>     at Object.<anonymous> (…\dist\servidor.js:13:22)
> Emitted 'error' event on Server instance at: …
>   code: 'EADDRINUSE', errno: -4091, syscall: 'listen', address: '::', port: 3010
> ```
>
> **Tarea:** 1) indica las 3 causas raíz más probables, ordenadas, con el comando exacto de PowerShell para confirmar o descartar cada una; 2) explica por qué el proceso termina con un stack trace en lugar de un mensaje claro; 3) propone el cambio mínimo en `src/servidor.ts` para que este fallo sea diagnosticable sin ocultar el error. **Restricción:** no sugieras terminar procesos de forma indiscriminada (por ejemplo, todos los `node.exe`)."
>
> Texto de la respuesta tal como la generó la IA (sin editar).

---

1. Causas raíz más probables

Causa 1 (más probable): la instancia anterior de VetSync sigue viva. En Windows, detener el proceso padre (la tarea, la consola o `npm`) no siempre termina al proceso hijo `node`: no hay señales POSIX que se propaguen al árbol de procesos. El `node dist/servidor.js` anterior queda huérfano y conserva el puerto.

```
$c = Get-NetTCPConnection -LocalPort 3010 -State Listen
Get-CimInstance Win32_Process -Filter "ProcessId=$($c.OwningProcess)" | Select-Object ProcessId,
ExecutablePath, CommandLine
```

Si la línea de comandos es `node dist/servidor.js` (o `ts-node src/servidor.ts`), es esta causa. Se termina **ese PID**, no todos los `node.exe`: `Stop-Process -Id <PID>`.

Causa 2: otra aplicación usa el puerto 3010. El mismo comando mostrará otro ejecutable (otro servidor de desarrollo, un proxy). Solución: cambiar `PUERTO` en `.env`.

Causa 3 (poco probable aquí): el puerto está en un rango reservado por Windows. Hyper-V, WSL y Docker Desktop reservan rangos de puertos dinámicamente.

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

Este caso suele producir `EACCES` (permiso denegado) y no `EADDRINUSE`, por eso queda en tercer lugar.

2. Por qué termina con un stack trace

`app.listen()` devuelve un `http.Server`, que es un `EventEmitter`. Cuando el `listen` falla, Node emite el evento `'error'`. Un `EventEmitter` que emite `'error'` **sin ningún listener** lanza la excepción: de ahí el `Unhandled 'error' event`. Como ocurre fuera de cualquier `try/catch` (es asíncrono), el proceso muere con el stack trace crudo. Además, en el código actual el `setInterval` del job de recordatorios se programa antes de saber si el servidor arrancó.

3. Cambio mínimo propuesto en `src/servidor.ts`

```
const servidor = app.listen(config.puerto, () => {
  registro.info(`VetSync escuchando en http://localhost:${config.puerto} (persistencia:
${config.persistencia})`);
});

servidor.on('error', (error: NodeJS.ErrnoException) => {
  if (error.code === 'EADDRINUSE') {
    registro.error(
      `El puerto ${config.puerto} ya está en uso. ¿Hay otra instancia de VetSync abierta? ` +
        `Revísalo con: Get-NetTCPConnection -LocalPort ${config.puerto} -State Listen`
    );
  } else {
    registro.error('No se pudo iniciar el servidor HTTP', error);
  }
  process.exit(1);
});
```

El error no se oculta: se registra con una pista accionable y el proceso termina con código 1, para que un supervisor o la CI lo detecten como fallo.

A.4 Respuesta de la IA — Prompt 4: Modelado de amenazas STRIDE + OWASP API Top 10

> **Herramienta:** Claude Code (modelo Claude Opus 5.5), en terminal, con acceso al repositorio. **Prompt exacto:**
>
> "Actúa como especialista en Seguridad de Aplicaciones (AppSec). **Contexto:** VetSync es una API REST (Express 4 + MySQL) que maneja datos personales de dueños y el historial médico de sus mascotas. Por decisión de alcance de la práctica no tiene autenticación de usuarios. El código está en `src/` y las rutas en `src/modulos/*/http`. **Tarea:** realiza un modelado de amenazas STRIDE de la API y relaciona cada amenaza con el OWASP API Security Top 10 (2023). **Formato de salida:** una tabla priorizada con: amenaza, categoría STRIDE, riesgo OWASP API, evidencia (archivo y fragmento de código que la origina), severidad y mitigación. Después, separa (a) mitigaciones implementables hoy sin dependencias nuevas y en pocas líneas, y (b) las que requieren una decisión de producto. **Restricciones:** no reportes ninguna vulnerabilidad sin señalar el código que la origina; marca como 'no aplica' las categorías del Top 10 que no correspondan y explica por qué."
>
> Texto de la respuesta tal como la generó la IA (sin editar).

Tabla priorizada

#	Amenaza	STRIDE	OWASP API 2023	Evidencia	Severidad	Mitigación
1	Cualquiera lee el historial médico de cualquier mascota recorriendo ids secuenciales	Information disclosure	API1 Broken Object Level Authorization	`rutasMascotas.ts`: `GET /:id/historial` sin verificación de dueño; ids `AUTO_INCREMENT`	Alta	(b) autenticación + autorización por dueño
2	Cualquiera cancela la cita de otra persona	Tampering	API1 BOLA	`rutasCitas.ts`: `PATCH /:id/cancelar`	Alta	(b)
3	Cualquiera dispara el job de recordatorios y lee **todos** los correos enviados (emails de todos los dueños)	Elevation of privilege / Information disclosure	API5 Broken Function Level Authorization	`rutasNotificaciones.ts`: `POST /recordatorios/ejecutar` y `GET /` sin ninguna protección	Alta	**(a)** token de administrador por cabecera
4	No hay identidad: no se puede saber quién creó o canceló una cita	Spoofing / Repudiation	API2 Broken Authentication	No existe middleware de autenticación en `app.ts`	Alta	(b)

#	Amenaza	STRIDE	OWASP API 2023	Evidencia	Severidad	Mitigación
5	Inyección de cabeceras de correo: un nombre con `\r\n` se copia al asunto	Tampering	API8 Security Misconfiguration / inyección	`validacion.ts`: `textoRequerido` sólo hace `trim()`; `servicioRecordatorios.ts`: `asunto: \`Recordatorio: cita de ${contacto.nombreMascota}\``	Media (hoy el correo es simulado; crítica al conectar SMTP)	**(a)** rechazar caracteres de control en textos de una línea
6	Respuestas con datos médicos sin cabeceras de seguridad; un navegador o proxy puede cachearlas o interpretarlas como HTML	Information disclosure	API8 Security Misconfiguration	`app.ts`: sólo `app.disable('x-powered-by')`	Media	**(a)** `nosniff`, `Cache-Control: no-store`, CSP restrictiva
7	Sin límite de peticiones: fuerza bruta de ids y saturación	Denial of service	API4 Unrestricted Resource Consumption	`app.ts` no tiene rate limiting (sí límite de cuerpo de 100 kb y listados acotados)	Media	(b) rate limiting en el proxy o con `express-rate-limit`
8	Un actor reserva todos los horarios de un veterinario	Denial of service	API6 Unrestricted Access to Sensitive Business Flows	`servicioCitas.ts`: no hay tope de citas programadas por mascota o dueño	Media	(b) regla de negocio (p. ej. máx. 3 citas futuras por mascota)
9	API sin versión en la ruta	—	API9 Improper Inventory Management	`app.ts`: prefijo `/api` sin versión	Baja	(b) `/api/v1`

No aplica: API3 (Broken Object Property Level Authorization): las respuestas no incluyen propiedades internas sensibles más allá de las ya cubiertas en #3. API7 (SSRF): la API no hace peticiones salientes a URLs del usuario. API10 (Unsafe Consumption of APIs): no consume APIs de terceros. Inyección SQL: todas las consultas usan parámetros `?`.

(a) Implementables hoy, sin dependencias nuevas

1. **Token de administrador** (#3): variable `TOKEN_ADMIN`; las rutas de notificaciones exigen la cabecera `X-Token-Admin` y la comparan con `crypto.timingSafeEqual` para no filtrar el token por tiempos de respuesta. Si `TOKEN_ADMIN` no está configurado, esas rutas responden 401 siempre (cerradas por defecto).
2. **Caracteres de control** (#5): `textoRequerido` rechaza `\u0000–\u001F` y `\u007F` en campos de una línea; los campos multilínea (`tratamiento`, `notas`) permiten `\n` y `\t`.
3. **Cabeceras de seguridad** (#6): middleware que agrega `X-Content-Type-Options: nosniff`, `Cache-Control: no-store`, `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'` y `Referrer-Policy: no-referrer`.

(b) Requieren decisión de producto

Autenticación (JWT u OAuth) con roles dueño / veterinario / administrador (#1, #2, #4), rate limiting (#7), reglas anti-acaparamiento de horarios (#8) y versionado de la API (#9).

A.5 Respuesta de la IA — Prompt 5: Contrato OpenAPI 3.1 a partir del código

> **Herramienta:** Claude Code (modelo Claude Opus 5.5), en terminal, con acceso al repositorio. **Prompt exacto:**
>
> "Actúa como Arquitecto de APIs especializado en diseño *contract-first*. **Contexto:** VetSync es una API Express; las rutas están en `src/modulos/*/http`, la validación de entrada en `src/compartido/http.ts` y `src/compartido/validacion.ts`, los errores en `src/compartido/errores.ts`, la seguridad en `src/compartido/seguridad.ts` y las reglas de citas en `src/modulos/citas/dominio/politicaAgendamiento.ts`. **Tarea:** genera la especificación OpenAPI 3.1 completa (`docs/openapi.yaml`) que describa exactamente el comportamiento implementado: rutas, parámetros, cuerpos con tipos y restricciones (longitudes, rangos, formatos, enumeraciones), respuestas con todos sus códigos, el esquema de error común y el esquema de seguridad de las rutas de administración. **Restricciones:** no documentes campos ni códigos de estado que el código no implemente; si el código y `docs/API.md` discrepan, gana el código y enumera las discrepancias al final. **Formato de salida:** YAML válido con componentes reutilizables en `components`."
>
> Texto de la respuesta tal como la generó la IA (sin editar).

`docs/openapi.yaml`

```
  /citas:
    post:
      tags: [Citas]
      operationId: agendarCita
      summary: Agenda una cita si el veterinario está libre
      description: >-
        Hay choque si existe otra cita PROGRAMADA del mismo veterinario con inicio < nuevoFin y
        fin > nuevoInicio. Las citas contiguas están permitidas. La verificación es atómica.
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CitaNueva' }
      responses:
        '201':
          description: Cita programada
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Cita' }
        '400': { $ref: '#/components/responses/Validacion' }
        '404': { $ref: '#/components/responses/NoEncontrado' }
        '409': { $ref: '#/components/responses/Conflicto' }
        '500': { $ref: '#/components/responses/Interno' }
  /citas/{id}:
    get:
      tags: [Citas]
      operationId: obtenerCita
      summary: Obtiene una cita
      parameters: [{ $ref: '#/components/parameters/Id' }]
      responses:
        '200':
          description: Cita
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Cita' }
```

```
        '400': { $ref: '#/components/responses/Validacion' }
        '404': { $ref: '#/components/responses/NoEncontrado' }
        '500': { $ref: '#/components/responses/Interno' }
```

Fragmento: líneas 216–253 de 495. El texto completo de esta respuesta está en el repositorio: `docs/bitacora/respuestas/12-contrato-openapi.md`.

Discrepancias entre `docs/API.md` y el código (gana el código)

1. `API.md` no documenta que `/notificaciones` y `/notificaciones/recordatorios/ejecutar` exigen la cabecera `X-Token-Admin`, ni las respuestas **401** y **403**.
2. La tabla de errores de `API.md` no incluye `ErrorNoAutorizado` (401) ni `ErrorProhibido` (403).
3. `API.md` no menciona el **horizonte de 365 días** para `inicio` de una cita.
4. `API.md` no indica que la fecha debe **existir en el calendario** (se rechaza `2027-02-30`).
5. `API.md` no indica que los enteros del cuerpo deben ser **números JSON** (se rechaza `"30"`), aunque en la ruta (`/citas/5`) se acepte texto.
6. `API.md` no menciona que los textos de una línea **no admiten caracteres de control**, y que `tratamiento` y `notas` sí admiten saltos de línea y tabuladores.

Resultado de validar la especificación generada

```
$ npx @redocly/cli@1 lint docs/openapi.yaml
…
Warning was generated by the tag-description rule.        (6 etiquetas sin descripción)
Operation must have at least one `4XX` response.          (GET /salud, GET /veterinarios)
docs\openapi.yaml: validated in 139ms
❌ Validation failed with 14 errors and 9 warnings.
```